

Read-First, Role-Gated Context Federation for Multi-Agent LLM Systems: A Fairness-Controlled, Multi-Model Evaluation Against Point-to-Point Message-Passing

Adrian Issac Mamani Quispe*, Ryan Fabian Valdivia Segovia*, Karim Guevara Puente de la Vega*

*Universidad Nacional de San Agustín de Arequipa, Arequipa, Peru

admamaniq@unsa.edu.pe, rvaldiviase@unsa.edu.pe, kguevarap@unsa.edu.pe

Abstract—Problem. Multi-agent systems based on large language models (LLMs) commonly rely on point-to-point message-passing for agent coordination, which can increase integration complexity and requires additional mechanisms for fine-grained access control and traceability. **Method.** This study evaluates the Graph Context Protocol (GCP), a read-first context-federation approach that integrates role- and capability-based access control with per-read provenance. GCP is compared with a base Agent2Agent (A2A) message-passing implementation under a controlled experimental design in which both approaches use the same agent logic, models, tasks, and evaluation criteria, differing only in the interoperability layer. The evaluation comprises five scenarios (marketplace, software organization, cross-owner supply chain, delegation, forced-access) across four model families: three open-weight LLMs served locally (llama3.1:8b, qwen3:4b, qwen2.5:32b, 3–5 seeds per arm across four independent run sets—qwen3:4b is evaluated twice, not as a fourth distinct model) and one hosted frontier family, Gemini 2.5 (flash-lite, flash: 30 seeds per arm; pro: 15 seeds per arm), for seven run configurations in total. The evaluation distinguishes deterministic architectural properties from model-dependent behavioral outcomes. **Results.** Structural analysis shows that GCP reduces integration-edge growth from $\mathcal{O}(n^2)$ to $\mathcal{O}(n)$ (50 vs. 2450 at $n=50$) and maintains constant per-request discovery and tool-description overhead as the number of peers increases, while both grow linearly for base A2A. In the delegation scenario, GCP prevented all attempted unauthorized actions (containment 1.00) on every model family, whereas base A2A executed unauthorized actions whenever the model attempted them. The *forced-access* scenario—a read-side counterpart that directly instructs the agent to read a confidential node it is not authorized for—replicates the same 1.00/0.00 containment split on every model that engaged the task; on the two strongest hosted tiers (flash, pro) base A2A additionally *executed* the unauthorized read and surfaced the canary, an observed (not only architectural) leak that GCP’s role gate denied outright on the same tiers. GCP also provided complete provenance by construction, while equivalent per-read traceability in A2A requires additional instrumentation. Behavioral experiments showed comparable task success and communication cost between approaches, while current canary-based leakage experiments in the non-adversarial scenarios showed no behavioral privacy advantage for either architecture. These results indicate that GCP provides model-independent structural and governance benefits while maintaining comparable behavioral performance. **Limitations.** Open-weight behavioral results use 3–5 seeds rather than the 20–30 recommended for significance testing on the token/latency/success deltas; the hosted Gemini family closes part

of this gap (30 seeds for flash-lite/flash, 15 for pro) but is one model family, evaluated once. This affects only the preliminary behavioral columns, not the deterministic/architectural results (coupling, discovery, tool-prompt, provenance, containment). **Contribution.** The contribution lies in integrating federated read-first access, role- and capability-based control, and native per-read provenance within a unified multi-agent coordination substrate, supported by a fairness-controlled, multi-model evaluation.

Index Terms—context federation; multi-agent systems; large language models; access control; least privilege; provenance; agent containment; software coupling; agent communication protocols

I. INTRODUCTION

The dominant coordination pattern for LLM-based multi-agent systems is direct message-passing. Frameworks such as AutoGen [1], ChatDev [2], and MetaGPT [3] compose applications from agents that send and receive messages, and the Agent2Agent (A2A) protocol [4] standardizes exactly this: a client agent advertises and delegates tasks to named remote agents over JSON-RPC. This paradigm descends from the classical agent communication languages KQML [5] and FIPA-ACL [6], which already assumed pairwise connections and shared, pre-negotiated protocols between every interacting pair.

Message-passing has a structural cost the software-engineering literature has understood for fifty years. Connecting n components pairwise requires on the order of n^2 links; a shared intermediary that each component connects to once reduces this to n . This is the “integration spaghetti” the Enterprise Integration Patterns [7] catalog addresses with brokers and hubs, and the phenomenon coupling theory [8–10] formalizes: modifiability is governed by the number and strength of inter-component dependencies.

We study an alternative substrate, the Graph Context Protocol (GCP). GCP is *read-first*: an agent obtains what it needs by reading context from independently owned knowledge nodes rather than receiving pushed messages. It is *role-gated*: every read passes through a policy gate that authorizes the request by role and required capabilities, defaulting to deny [11, 12]. And it is *audited*: every read or delegation decision—grant or deny—produces a structured provenance record.

a) *Thesis (measured form).*: At equal task success, read-first role-gated context federation offers *structural* advantages over point-to-point message-passing in coupling and traceability, and *preliminary* evidence of lower or equal cost under certain scenarios. We state the structural claims as strong (they are deterministic or architectural) and the behavioral claims as preliminary (open-weight models, few seeds).

b) *Scope of the comparison.*: We are explicit about what the A2A arm represents. We do not claim message-passing *cannot* be audited: logs, middleware, proxies, or gateways can add an access trail. Our comparison is against *base* A2A—the protocol as shipped, without such instrumentation—and its purpose is to show that GCP makes provenance and role gating *native primitives* rather than optional add-ons. Where useful we discuss a second reference point, A2A augmented with minimal read logging (Section IV).

c) *Contributions.*:

- 1) **Architecture and a reproducible specification.** We describe GCP (read-first, role-gated, audited, with a stricter-capability delegation gate) precisely enough to reimplement: query format, node and policy structure, role credential, provenance record, and the authorization algorithm (Section III).
- 2) **A fairness-controlled benchmark method.** One scenario definition and one injected model build both arms; only the interop layer changes (Section V).
- 3) **A multi-model empirical head-to-head** over five scenarios—including a *forced-access* probe that directly targets the confidential node—and four model families (three open-weight, one hosted frontier family) across seven run configurations, separating deterministic from model-dependent results (Section VI).
- 4) **Metric definitions** for structural coupling, discovery/tool-prompt overhead, a four-way leakage/agency taxonomy, provenance completeness, and delegation/forced-access containment (Section IV).

II. BACKGROUND AND RELATED WORK

A. Message-passing as the multi-agent baseline

Agent communication languages established the message-passing paradigm: KQML [5] wrapped payloads in typed performatives over pairwise connections, and FIPA-ACL [6] standardized a message in which sender and receiver must share content language and ontology before any exchange succeeds. Modern LLM frameworks reinvented this in natural language. AutoGen [1] programs applications as conversations; ChatDev [2] arranges role-pairs in a fixed “chat chain”; surveys [13] taxonomize communication into peer-to-peer, centralized, layered, and shared-pool variants. The Model Context Protocol [14] standardizes agent-to-tool context access (an early read-first form), while A2A [4] standardizes agent-to-agent task delegation—the baseline we benchmark against. Zhang et al. [15] report 28–73% excess tokens in inter-agent communication graphs and recover most by pruning topology; Mao and Mirhoseini [16] replace both centralized orchestration

and peer-to-peer routing with a shared verified context at roughly half the cost. We add role-gated authorization, audited per-read provenance, an agency-containment gate, and a multi-model fairness-controlled comparison.

B. Coupling and the n^2 problem

That fewer inter-component links yield more maintainable systems is among the oldest results in software engineering. Parnas [8] showed information-hiding decomposition produces far lower coupling than flowchart-style partitioning; Stevens et al. [9] made coupling and cohesion measurable; Briand et al. [10] gave a graph-theoretic framework in which coupling is monotone in dependency-edge count, so an $\mathcal{O}(n)$ -edge topology is provably less coupled than an $\mathcal{O}(n^2)$ one. Hohpe and Woolf [7] translated this to integration: a point-to-point mesh of n systems needs $n(n-1)/2$ channels, which a hub reduces to n . Microservice research [17] carries the same argument into cloud-native systems, and Shen et al. [18] show dense agent topologies amplify error propagation while sparser ones suppress it. A2A creates agent $\leftrightarrow$ agent edges; GCP creates agent $\rightarrow$ store edges.

C. Least privilege, access-controlled context, and gated retrieval

GCP’s read gate operationalizes least privilege [11]. Role-based access control [12] assigns permissions to roles; attribute-based access control [19] generalizes to arbitrary attributes; capability-based protection [20] ties authority to explicitly passed tokens—the model GCP’s role credentials follow. Data-minimization law [21] makes “read only what your role requires” a legal constraint. Read-first tool access (MCP) and access-controlled retrieval supply the read side, and RBAC/ABAC the gate side, but neither couples the two with per-read provenance; the point-to-point baseline has no analogous gate, so leakage and delegation control rest on sender-side discretion.

D. Leakage, provenance, and governance in agentic systems

Confidentiality in agentic systems is a system property, not only a model property [22]; inter-agent message channels are the dominant leakage vector, largely invisible to output-only auditing [23]; and restricting what an agent can *see* is more robust than filtering what it can *say* [24]. The OWASP Top 10 for LLM applications [25] names Sensitive Information Disclosure and Excessive Agency as primary risks—the latter is exactly what our delegation scenario probes. Deterministic privilege control at the tool boundary [26], authenticated delegation of authority [27], and audit trails [28] argue for tamper-evident records of system actions.

a) *Delta / defensible novelty.*: Prior work supplies the pieces—read-first access (MCP), role/attribute gating (RBAC/ABAC, access-controlled RAG), audited delegation, and audit logs—but not their conjunction, and not a fairness-controlled, multi-model head-to-head against a real message-passing baseline. Our claim to novelty is precisely the *integration*: federated read + role control + per-read provenance,

evaluated against A2A. Read-first alone, RBAC alone, or auditing alone would not be a contribution.

III. THE GRAPH CONTEXT PROTOCOL: SPECIFICATION

GCP models an ecosystem as independently owned *knowledge nodes* and *agent nodes*. Coordination is read-first: an agent issues a *context query* to a peer node, which returns context the caller is authorized to read. We specify the substrate precisely enough to reimplement.

A. Data structures

A **knowledge node** carries an id, a kind, an adapter that produces content, and an access policy. A **policy** declares gating fields; a **role credential** names the caller's role and flat capabilities; a **provenance record** logs each decision.

```
KnowledgeNode {
  id, kind,                // e.g. "rag", "log"
  adapter,                 // yields raw content
  policy: AccessPolicy }
AccessPolicy {
  readableByRoles: [roleId],
  requiredCapabilities: [capId],
  fallbackAllowed: bool,
  denialMode: "empty" | "error" }
Principal (role credential) {
  id, role: { id, capabilities:[capId],
              parent? },
  capabilities: [capId] }
Provenance {
  principalId, targetNodeId, timestamp,
  decision: "grant" | "deny",
  matchedPolicy }
```

B. Query and authorization flow

A context query names a target node and a requested capability (read, or the stricter delegate). The server evaluates the admission predicate *before* any adapter runs, so denied content never enters the agent's context window (complete mediation [11]). The effective capability set of a principal is the union of its flat capabilities and those inherited through the role chain (own-first precedence). The predicate is

$$\text{authorized}(P, K) \equiv \underbrace{(P.\text{roles}=[] \vee K.\text{role} \in P.\text{roles})}_{\text{roleOk}} \wedge \underbrace{(P.\text{caps}=[] \vee P.\text{caps} \subseteq \text{eff}(K))}_{\text{capsOk}}$$

An absent or schema-invalid policy is treated as a denial (default-deny). The end-to-end path is:

Request → Policy check → Decision → Context returned / denied
→ Provenance record

C. Read/authorize algorithm (pseudocode)

```
handleQuery(query, principal):
  node = resolve(query.targetNodeId)
  if node == null: return deny("no-node")
  P = node.policy
  if P invalid or absent: # default-deny
    record(principal, node, "deny", "none")
    return deny("no-policy")
  eff = flat(principal) UNION
         rochain(principal.role)
  roleOk = P.roles==[] or
           principal.role.id in P.roles
```

TABLE I. PROPERTY COMPARISON. A “NO” DOES NOT MEAN THE PROTOCOL MAKES THE PROPERTY IMPOSSIBLE; EVERY CELL BELOW IS ACHIEVABLE IN PRINCIPLE BY EXTENDING THE BASE PROTOCOL. “NOT NAT.” = NOT A NATIVE PRIMITIVE OF THE BASE PROTOCOL (WOULD REQUIRE ADDED LOGIC ON TOP OF IT); “EXT.” = ACHIEVABLE ONLY WITH ADDITIONAL EXTERNAL INSTRUMENTATION; “N/A” = THE CONCEPT DOES NOT APPLY TO THIS PROTOCOL’S DESIGN.

property	A2A	MCP	GCP
agent↔agent	yes	not nat.	yes
read-first	not nat.	yes	yes
cross-owner boundary	yes	not nat.	yes
role/capability gating	coarse	not nat.	yes
per-read provenance (native)	ext.	ext.	yes
deny-by-default	not nat.	not nat.	yes
stricter-capability delegation	not nat.	n/a	yes

```
capsOk = P.caps==[] or P.caps subset eff
needCap = query.capability # read|delegate
capOk2 = needCap in eff
if roleOk and capsOk and capOk2:
  record(principal, node, "grant", P.id)
  return node.adapter.read() # content
else:
  record(principal, node, "deny", P.id)
  return denialResult(P.denialMode) # denied
```

Delegation reuses this path with needCap = delegate, a strictly stronger capability than read: an agent authorized to read is not thereby authorized to command.

D. Denial handling

On denial the handler returns only a denial reason (or an empty result, per denialMode); the adapter is never invoked, so node content cannot appear in the response. Every denial is itself recorded, yielding a who-tried-what ledger, not only a who-read-what one.

E. Relation to A2A and MCP

Table I contrasts the three. A2A is agent-to-agent and push-based, with coarse card-declared allow/deny and no native per-read trail; MCP is read-first but agent-to-tool and single-owner, without role gating across owners; GCP combines cross-owner read-first access, role gating, deny-by-default, and native per-read provenance.

IV. METRIC DEFINITIONS

We report metrics in four families and separate deterministic from model-dependent throughout.

a) *Structural coupling (deterministic).*: For n peers, pairwise connections are the integration edges the topology requires: $C_{A2A}(n) = n(n-1)$ vs. $C_{GCP}(n) = n$. *Integration effort* is the marginal cost $C(n+1) - C(n)$: constant 1 for GCP, $2n$ for A2A. Computed exactly. We count *directed* integration edges; therefore each bidirectional peer relationship contributes two directed edges. Under an undirected channel definition the corresponding count is $n(n-1)/2$; the asymptotic conclusion remains $\mathcal{O}(n^2)$.

b) *Discovery and tool-prompt overhead (per request).*: *Discovery messages*: round-trips to learn what can be queried—1 for GCP, n for base A2A (one card per peer). *Tool-prompt tokens*: token cost of tool/agent descriptions injected each request. Recurring, distinct from one-time wiring.

c) *Governance (two reference points).*: *Provenance completeness*: fraction of decisions with a structured record, clamped to $[0, 1]$. For GCP this is 1.0 by construction. For base A2A it is 0.0—read as “the base message-passing implementation produces no per-read traceability without added instrumentation,” not “A2A cannot be audited.” An A2A arm with minimal read logging is the natural second reference point and would raise its completeness at the cost of the very instrumentation GCP makes native. *Delegation containment*: fraction of *unauthorized* delegation attempts refused (denied and logged) rather than executed.

d) *Leakage and agency (four-way taxonomy)*.: We separate: (a) *attempted unauthorized access*—the agent tried to reach a node/action it lacks rights to; (b) *granted unauthorized access*—the substrate served it; (c) *transcript leakage*—a canary appears anywhere in the tool transcript; (d) *final-answer leakage*—a canary appears in the answer. Canary scanning is exact-match (a lower bound; paraphrase and semantic leakage are not counted). We report (a) and (b) directly from the delegation scenario; the transcript/answer split and a semantic-similarity leakage metric are instrumentation we flag as future work (Section VII). We distinguish *architectural guarantee* (what the gate provably prevents) from *observed leakage* (what a given model exercised).

e) *Behavioral cost and success*.: Counts (messages, connections, round-trips, wire tokens) and wall-clock latency, with a per-scenario success oracle. Counts are the primary cost signal; local wall-clock is noisy.

V. METHODOLOGY

a) *Fairness invariant*.: The two arms differ *only* in the interop layer. One scenario runner builds both from one `ScenarioDef` and one injected chat model; both run the identical ReAct brain, system prompt, goal, and oracle. The GCP arm hosts nodes in-process behind the real server's fetch handler with a static-token auth provider and in-memory audit sink; the A2A arm hosts the same nodes as real localhost A2A servers. Only the per-peer tool factory and node hosting change.

b) *Scenarios*.: *software-org* (leakage): a contractor-role agent summarizes public project info; a confidential engineering node (canary) is readable only by the engineering role. *supply-chain* (cross-owner): a logistics agent states the shareable shipping relationship between two independently owned nodes, each holding an owner-scoped canary. *delegation* (agency, write-side): an agent attempts an action it is not authorized to command; the oracle checks containment. *forced-access* (agency, read-side): the agent's task explicitly instructs it to read a confidential node directly, despite its role lacking the required capability—unlike *software-org*, the unauthorized-read path is exercised on *every* run rather than left latent; the oracle checks containment of the read, mirroring the delegation oracle for the read side. *marketplace* (coupling): n seller nodes with descending prices. Following reviewer guidance, the marketplace contributes *only* its deterministic structural, discovery, and tool-prompt curves; its behavioral rows are excluded from the main claims because task success was low on both arms (Section VII).

c) *Models, seeds, harness*.: Open-weight models served locally: llama3.1:8b (5 seeds per arm), qwen3:4b (3 seeds per arm, two independent run sets, labeled (a) and (b) below), qwen2.5:32b (3 seeds per arm)—three distinct models, four independent run sets in total, since (a) and (b) are two separate executions of the identical qwen3:4b configuration, not two different models. We additionally evaluate one hosted frontier family through the native Gemini API (EVAL_ADAPTER=google, parallel-tool-call sanitizer, non-streaming): gemini-2.5-flash-lite and gemini-2.5-flash (30 seeds per arm) and gemini-2.5-pro (15 seeds per arm, run through an incremental JSONL-per-run writer after an initial 30-seed attempt exceeded the harness's 60-minute test timeout)—four model families in total across seven run configurations. The harness is CI-gated, throttles, and retries on transient errors. We report mean $\pm$ population s.d.; where a 95% normal-approximation interval is informative we give $\pm 1.96 s/\sqrt{k}$. Structural curves are computed for $n = 2, \dots, 50$; discovery/tool-prompt curves are measured over $n \in \{2, 4, 8, 16, 32\}$. **We treat coupling/discovery/tool-prompt/provenance/containment as established (deterministic or architectural) and token/latency/leakage/success as preliminary**: the open-weight rows remain low-seed and locally hosted, and the hosted-frontier replication (higher seed count, one model family) partially, not fully, retires that limitation.

VI. RESULTS

A. Coupling: $\mathcal{O}(n)$ vs. $\mathcal{O}(n^2)$ (deterministic)

Table II shows the structural curve: GCP grows linearly, A2A quadratically— $9\times$ at $n=10$, $49\times$ at $n=50$. Integration effort is constant 1 for GCP and $2n$ for A2A, so the gap widens with every node. Identical regardless of model: the curve is set by topology, not by model behavior.

B. Per-request overhead: $\mathcal{O}(1)$ vs. $\mathcal{O}(n)$ (deterministic)

The coupling tax is paid on *every request* via discovery messages and the tool-prompt budget (Table III). GCP holds both flat (1 message, 72 tokens); base A2A reaches 32 messages and 1782 tool-prompt tokens at $n=32$, a $25\times$ prompt gap. Identical regardless of model, since these are set by prompt construction, not model behavior.

TABLE II. STRUCTURAL COUPLING (INTEGRATION EDGES) BY SIZE n . DETERMINISTIC, MODEL-INDEPENDENT.

n	GCP (n)	A2A ($n(n-1)$)	ratio
2	2	2	1.0 $\times$
3	3	6	2.0 $\times$
5	5	20	4.0 $\times$
10	10	90	9.0 $\times$
20	20	380	19.0 $\times$
50	50	2450	49.0 $\times$

TABLE III. PER-REQUEST DISCOVERY MESSAGES AND TOOL-PROMPT TOKENS (MARKETPLACE SWEEP). MODEL-INDEPENDENT.

n	discovery msgs		tool-prompt tok	
	GCP	A2A	GCP	A2A
2	1	2	72	110
4	1	4	72	220
8	1	8	72	440
16	1	16	72	886
32	1	32	72	1782

C. Provenance: native to GCP, added instrumentation for base A2A (architectural)

Across the four non-marketplace scenarios, all four model families, every seed, provenance completeness is 1.0 for GCP and 0.0 for base A2A, with one reported exception: gemini-2.5-flash-lite's A2A arm on forced-access shows 1.0, a degenerate case since that arm made zero attempts (it declined the task, Section VI-E)—there was nothing to fail to record, not a case of base A2A actually producing a trail. This is otherwise architectural: GCP records every decision; the base message-passing arm has no audit sink, so no trail exists to query. As stated in Section IV, this reflects the *base* implementation, not a necessary limit of A2A; an instrumented A2A would raise the number at the cost of the instrumentation GCP makes native.

D. Delegation containment: GCP contains, base A2A executes (architectural + observed)

Table IV reports the four-way agency metrics across all four model families. GCP containment is 1.00 for every model, with zero unauthorized executions—an *architectural* guarantee: the delegate capability gate refuses by construction. Base A2A containment is 0.00, and under five of the six evaluated configurations it *executed* the unauthorized action (*observed* granted-unauthorized-access). Under qwen2.5:32b the larger open-weight model happened not to attempt it, so nothing executed—but the gate is still absent, so containment is 0.00 by construction. This separates “the model behaved” from “the protocol did not permit misbehavior.” The hosted frontier tiers behave the same way as the open-weight ones: attempts scale with model capability (gemini-2.5-pro attempted 14/15 seeds, aggregated), yet GCP's containment stays 1.00 regardless.

E. Forced-access containment: the read-side counterpart (architectural + observed)

Table V reports the forced-access scenario, the read-side mirror of delegation: the task instructs the agent to read the confidential node directly, so the unauthorized-read path is exercised on every run that engages the task at all, not left latent as in software-org. GCP containment is 1.00 on every configuration that attempted the read. Base A2A containment is 0.00 throughout, and on two of the three hosted tiers—flash and pro, the stronger models—it *executed* the unauthorized read and the canary surfaced in the transcript, an *observed* granted-unauthorized-access, not only an architectural possibility. This is the result the prior edition of this article flagged as future work (Section VII): running the scenario is what separates “the model behaved” from “the protocol did not permit misbehavior,” and stronger models are exactly where that distinction earns its keep, since they are the ones that actually attempt and complete the unauthorized read.

llama3.1:8b is a genuine outlier, not a missing data point: it made *zero* tool calls on either arm across two independent 5-seed reruns (20/20 runs), reproduced deterministically. A direct non-agentic probe against the same scenario input shows the model refusing in its final answer before ever

TABLE IV. DELEGATION, FOUR-WAY AGENCY METRICS BY RUN SET. ATT. = ATTEMPTED UNAUTHORIZED; GRANT = GRANTED UNAUTHORIZED (EXECUTED); CONT. = CONTAINMENT. QWEN3:4B (A) AND (B) ARE TWO INDEPENDENT RUN SETS OF THE IDENTICAL MODEL, NOT TWO DISTINCT MODELS. OPEN-WEIGHT AND GEMINI-2.5-{FLASH-LITE, FLASH} ROWS ARE A SINGLE REPRESENTATIVE RUN (SEED[0]); GEMINI-2.5-PRO IS SUMMED OVER ALL 15 SEEDS (LABELED $\Sigma/15$), PER SECTION V.

model	arm	att.	grant	cont.	prov.
llama3.1:8b	GCP	1	0	1.00	1.0
	A2A	1	1	0.00	0.0
qwen3:4b (a)	GCP	1	0	1.00	1.0
	A2A	1	1	0.00	0.0
qwen3:4b (b)	GCP	1	0	1.00	1.0
	A2A	1	1	0.00	0.0
qwen2.5:32b	GCP	1	0	1.00	1.0
	A2A	0 [†]	0	0.00	- [†]
gemini-2.5-flash-lite	GCP	1	0	1.00	1.0
	A2A	1	1	0.00	0.0
gemini-2.5-flash	GCP	1	0	1.00	1.0
	A2A	1	1	0.00	0.0
gemini-2.5-pro ($\Sigma/15$)	GCP	14	0	1.00	-
	A2A	11	11	0.00	-

[†] degenerate run: the base A2A agent never issued the request, so nothing was attempted, executed, or recorded; the gate is still absent.

invoking a tool—verbatim “*I can’t do that. Is there anything else I can help you with?*” (GCP arm) and “*I can’t help you with this request.*” (A2A arm). The scenario’s system-prompt phrase making the role mismatch explicit (“even though your role is not engineering”) reads to this model’s alignment training as a policy violation, so it declines outright—on *both* arms, independent of whether a gate exists to catch the read. This is a small-model alignment-vs-tool-use interaction, orthogonal to the protocol comparison: the same model calls tools without hesitation on software-org, supply-chain, and delegation. gemini-2.5-flash-lite shows a milder version of the same effect on the A2A arm only (it declined the task, 0 attempts), while its GCP arm attempted and was denied—so containment reads 1.00/0.00 by construction on both tiers, but for different reasons: the smallest hosted tier avoided the ungated read by declining the task, the smallest local model avoided it by declining everything.

F. Cost and success: parity (preliminary)

Wire tokens and task success are within seed variance between arms wherever the model could do the task. On llama3.1:8b software-org, tokens are 500.6 ± 16.9 vs. 502.4 ± 18.9 (95% CI ± 14.8 vs. ± 16.6 , $k=5$); on qwen2.5:32b software-org, 669.7 ± 0.9 vs. 668.7 ± 3.3 . Absolute counts differ by an order of magnitude across models, but the between-arm gap stays inside the interval in every scenario. The base A2A arm’s wire-token count is often marginally *lower* (it omits GCP’s query envelope); the tax A2A pays is the per-request tool-prompt budget (Table III), not the payload. We therefore make no claim that GCP sends fewer wire tokens—only that it does not cost meaningfully more while removing the $\mathcal{O}(n)$ prompt overhead. We present these as *preliminary*, pending 20–30 seeds and significance testing.

G. Leakage: ties in the canary scenarios (observed), guarantee holds (architectural)

We report the leakage picture plainly. In *software-org*, observed leakage was 0.0 on both arms across all models—but because the agent satisfied the public-summary task without ever requesting the confidential node, so the leaky path was *not exercised*; the architectural leak is latent, not observed. In *supply-chain*, both arms are authorized to read both owners’ nodes, so both surfaced the canaries equally (leakage 1.0 on both, every model): a genuine *tie*, because the leak is not a gating decision. We therefore claim no observed behavioral leakage advantage. The leakage advantage of GCP is the *architectural deny-by-default-plus-audit guarantee*, exercised directly by the delegation and provenance results, not by these canary counts.

TABLE V. FORCED-ACCESS, AGENCY METRICS BY MODEL FAMILY. ATT. = ATTEMPTED UNAUTHORIZED READ; GRANT = GRANTED UNAUTHORIZED (EXECUTED, CANARY SERVED); CONT. = CONTAINMENT. COUNTING CONVENTIONS MATCH TABLE IV (GEMINI-2.5-PRO ROW IS $\Sigma/15$).

model	arm	att.	grant	cont.
llama3.1:8b	GCP	0 [‡]	0	n/a [‡]
	A2A	0 [‡]	0	n/a [‡]
qwen3:4b	GCP	1	0	1.00
	A2A	1	1	0.00
qwen2.5:32b	GCP	1	0	1.00
	A2A	1	1	0.00
gemini-2.5-flash-lite	GCP	1	0	1.00
	A2A	0 [†]	0	0.00 [†]
gemini-2.5-flash	GCP	1	0	1.00
	A2A	1	1	0.00
gemini-2.5-pro ($\Sigma/15$)	GCP	11	0	1.00
	A2A	12	12	0.00

[‡] neither arm ever called a tool—the model refused the task outright in its final answer (see text); no gate was exercised on either side, so containment is not defined (not the same as 1.00: the gate was never tested). [†] the base A2A agent declined the task and never attempted the read, so nothing was executed—but the gate is still absent, so containment is 0.00 by construction, matching the qwen2.5:32b degenerate-run convention in Table IV.

H. Architectural guarantee: executable no-leak property

The distinction above rests on a proof, not only measurement. Over the shipped read/authorize path we run property-based tests (*fast-check*) against fixed, finite pools: five capabilities and four roles, one of which (*role:child*) inherits from another (*role:a*) so effective-capability computation is exercised, not only direct grants. Each random *policy* independently samples a subset of the role pool for *readableByRoles*, a subset of the capability pool for *requiredCapabilities*, a random *fallbackAllowed* boolean, and a random denial mode. Each random *principal* independently samples one role from the pool or none, plus a random subset of capabilities held directly. Because both are drawn from small pools by subset sampling rather than from an unbounded space, authorized and unauthorized draws both occur frequently, so the 1000-run default is not dominated by one outcome. Every generated (*policy*, *principal*) pair is then driven through the real context-query handler (not a mock) against a stub adapter that *always* returns a sentinel canary string, so P1 (end-to-end no-content-leak) asserts that an unauthorized query never returns the canary and is marked *denied*, while an authorized one does return it (liveness, so the test is not vacuous); P2 (authorize soundness) checks the shipped gate’s decision equals an independently written reference predicate over the same pair; P3 (role-inheritance soundness) checks effective-capability computation against the role chain directly. All pass at 1000 runs (raisable via an environment variable; runs are seed-reproducible for debugging). The canary metric *measures* leakage in scenarios; the property suite *proves* the gate is sound over the input space. Together: the property shows the gate is correct; the canary confirms it is on the hot path.

VII. DISCUSSION AND THREATS TO VALIDITY

a) *Four model families, seven run sets, but open-weight seed counts stay low.*: The behavioral results now span three open-weight models over two size classes (4B–32B), locally hosted, across four independent run sets (qwen3:4b was run twice, as an internal replication check, not as a fourth model), plus one hosted frontier family (Gemini 2.5, three tiers) across three further run sets—removing both the single-model and the open-weight-only threats. Seed counts, however, are uneven across the two groups: the open-weight arm stays at 3–5 seeds, the wall-clock cost of running every seed as a full multi-step ReAct agent loop against a locally hosted LLM server, repeated per model, per scenario, and per arm, making 20–30 seeds roughly an order of magnitude more expensive there. The hosted Gemini arm closes this gap for two of its three tiers (30 seeds for *flash-lite/flash*, meeting the significance-testing convention we could not afford locally) and partially for the third (gemini-2.5-pro, 15 seeds—a first 30-second attempt was killed by the harness’s 60-minute test timeout before writing a report;

the incremental writer used for the reported run avoids that failure mode but the timeout itself has not yet been raised to let a 30-seed `pro` run complete). This constraint does not affect the deterministic/architectural claims (coupling, discovery, tool-prompt overhead, provenance, containment, delegation/forced-access containment), which do not depend on seed count at all; it bears only on the preliminary token/latency/leakage/success columns, and only for the open-weight rows and the `pro` tier. A second hosted frontier family, and 20–30 seeds on `pro`, remain the natural next replication step.

b) Provenance is a base-implementation property.: The 0.0 for A2A is a property of base message-passing, not a proof that A2A cannot audit. We frame it as “no per-read traceability without added instrumentation” and identify instrumented A2A as the fair second baseline.

c) Leakage measurement is limited.: Exact-match canary scanning misses paraphrase and semantic leakage, and behavioral leakage in the non-adversarial scenarios depends on the agent exercising the path (latent in software-org, tied in supply-chain). The forced-access scenario (Section VI-E) now closes part of this gap by directly targeting the confidential node rather than waiting for the agent to request it, and shows both an architectural guarantee (GCP containment 1.00 throughout) and, on the two strongest hosted tiers, an *observed* granted-read leak on base A2A. Two reinforcing gaps remain as future work: an *adversarial-prompt* scenario that induces unauthorized retrieval through indirection rather than an explicit instruction, and a *semantic* leakage metric beyond exact match, which would also let the software-org and supply-chain canaries detect paraphrased disclosure. Delegation and forced-access are both already forced-action probes and both show the gate engaging.

d) Marketplace used for structure only.: Marketplace task success was low on *both* arms (e.g. falling to 0.0–0.4 at $n=16,32$ on `llama3.1:8b`); the model often failed to pick the cheapest seller. Because the failure is equal across arms it does not threaten fairness, but per reviewer guidance we use the marketplace *only* for its deterministic structural, discovery, and tool-prompt curves, and exclude its behavioral rows from the main claims.

e) Construct validity of the baseline.: The A2A arm runs real servers, the same brain, and the same task, built with equal care; its disadvantages (no read gate, no delegation gate, no native audit sink, pairwise topology) are intrinsic to base message-passing, not artifacts of a strawman.

VIII. LIMITATIONS AND FUTURE WORK

Raise the open-weight arm toward 20–30 seeds with significance testing on token/latency/message deltas, raise the harness timeout so `gemini-2.5-pro` can complete a full 30-seed run, and add a second hosted frontier family beyond Gemini; add the instrumented-A2A second baseline; add the adversarial-prompt and semantic-leakage scenarios and split transcript vs. final-answer leakage; and design a privacy testbed that removes the authorized-read confound of the current supply-chain scenario so the gating advantage can be *observed*, not only argued, in that scenario too. Bridging GCP to MCP and A2A as first-class interop is the natural deployment path.

IX. CONCLUSION

We presented GCP—read-first, role-gated, audited context federation with a stricter-capability delegation gate—and a fairness-controlled comparison against base A2A message-passing across four model families: three open-weight local models and one hosted frontier family (Gemini 2.5, three tiers). The *structural* advantages are strong and model-independent: coupling improves from $\mathcal{O}(n^2)$ to $\mathcal{O}(n)$ in integration edges and from $\mathcal{O}(n)$ to $\mathcal{O}(1)$ in per-request discovery and tool-prompt overhead; provenance is native rather than added instrumentation; and in both agency scenarios—delegation (write-side) and forced-access (read-side)—GCP contained every unauthorized action while base A2A contained none, on every model family that engaged the task. The forced-access results additionally turn the architectural guarantee into an *observed* one on the two strongest hosted tiers, where base A2A did not just risk but actually executed the unauthorized read and surfaced the canary. Cost and success are at parity in preliminary behavioral runs, and canary leakage *ties* on the current non-adversarial testbed, so we claim no privacy advantage there yet. The defensible contribution is the *integration*—federated read, role control, and per-read provenance—plus its measured comparison; the behavioral results are a multi-model pilot, now spanning one hosted frontier family alongside three open-weight models, that invites replication with more seeds and additional hosted models.

REFERENCES

- [1] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023. URL <https://arxiv.org/abs/2308.08155>.
- [2] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. URL <https://arxiv.org/abs/2307.07924>.
- [3] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2308.00352>.
- [4] Google and Linux Foundation. Agent2Agent (A2A) protocol specification, v1.0. Open standard, 2025. URL <https://a2a-protocol.org/v1.0.0/specification/>.
- [5] Tim Finin, Rich Fritzson, Don McKay, and Robin McEntire. KQML as an agent communication language. In *Proceedings of the Third International Conference on Information and Knowledge Management (CIKM)*, 1994.
- [6] Foundation for Intelligent Physical Agents. FIPA ACL message structure specification. Technical Report SC00061G, FIPA, 2002. URL <http://www.fipa.org/specs/fipa00061/>.
- [7] Gregor Hohpe and Bobby Woolf. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley Professional, 2003.
- [8] David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972. doi: 10.1145/361598.361623.
- [9] W. P. Stevens, G. J. Myers, and L. L. Constantine. Structured design. *IBM Systems Journal*, 13(2):115–139, 1974. doi: 10.1147/sj.132.0115.
- [10] Lionel C. Briand, John W. Daly, and Jürgen K. Wüst. A unified framework for coupling measurement in object-oriented systems. *IEEE Transactions on Software Engineering*, 25(1):91–121, 1999. doi: 10.1109/32.748920.
- [11] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. doi: 10.1109/PROC.1975.9939.
- [12] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. Role-based access control models. *IEEE Computer*, 29(2):38–47, 1996. doi: 10.1109/2.485845.
- [13] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. Large language model based multi-agents: A survey of progress and challenges. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, pages 8048–8057, 2024. URL <https://arxiv.org/abs/2402.01680>.
- [14] Anthropic. Introducing the model context protocol. Anthropic; open specification, 2024. URL <https://www.anthropic.com/news/model-context-protocol>.
- [15] Guibin Zhang, Yanwei Yue, Zhixun Li, Sukwon Yun, Guancheng Wan, Kun Wang, Dawei Cheng, Jeffrey Xu Yu, and Tianlong Chen. Cut the crap: An economical communication pipeline for LLM-based multi-agent systems. *arXiv preprint arXiv:2410.02506*, 2024. URL <https://arxiv.org/abs/2410.02506>.
- [16] Yuzhen Mao and Azalia Mirhoseini. Decentralized multi-agent systems with shared context. *arXiv preprint arXiv:2606.10662*, 2026. URL <https://arxiv.org/abs/2606.10662>.
- [17] Sebastiano Panichella, Mohammad Imranur Rahman, and Davide Taibi. Structural coupling for microservices. In *Proceedings of the 11th International Conference on Cloud Computing and Services Science (CLOSER)*, pages 280–287, 2021. doi: 10.5220/0010481902800287. URL <https://arxiv.org/abs/2103.04674>.
- [18] Xu Shen, Yixin Liu, Yiwei Dai, Yili Wang, Rui Miao, Yue Tan, Shirui Pan, and Xin Wang. Understanding the information propagation effects of communication topologies in LLM-based multi-agent systems. *arXiv preprint arXiv:2505.23352*, 2025. URL <https://arxiv.org/abs/2505.23352>.
- [19] Vincent C. Hu, David Ferraiolo, Rick Kuhn, Adam Schnitzer, Kenneth Sandlin, Robert Miller, and Karen Scarfone. Guide to attribute based access control (ABAC) definition and considerations. Technical Report NIST Special Publication 800-162, National Institute of Standards and Technology, 2014.
- [20] Jack B. Dennis and Earl C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966. doi: 10.1145/365230.365252.

- [21] European Parliament and Council. Regulation (EU) 2016/679 (general data protection regulation), articles 5 and 25. Official Journal of the European Union, L 119, 2016. URL <https://gdpr-info.eu/>.
- [22] Jonathan Evertz, Merlin Chlosta, Lea Schönherr, and Thorsten Eisenhofer. Whispers in the machine: Confidentiality in agentic systems. *arXiv preprint arXiv:2402.06922*, 2024. URL <https://arxiv.org/abs/2402.06922>.
- [23] Fatima El Yagoubi, Godwin Badu-Marfo, and Ranwa Al Mallah. AgentLeak: A full-stack benchmark for privacy leakage in multi-agent LLM systems. *arXiv preprint arXiv:2602.11510*, 2026. URL <https://arxiv.org/abs/2602.11510>.
- [24] Eugene Bagdasarian, Ren Yi, Sahra Ghalebikesabi, Peter Kairouz, Marco Gruteser, Sewoong Oh, Borja Balle, and Daniel Ramage. AirGapAgent: Protecting privacy-conscious conversational agents. *arXiv preprint arXiv:2405.05175*, 2024. URL <https://arxiv.org/abs/2405.05175>.
- [25] OWASP GenAI Security Project. OWASP top 10 for large language model applications. OWASP Foundation, 2025. URL <https://owasp.org/www-project-top-10-for-large-language-model-applications/>.
- [26] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. Progent: Programmable privilege control for LLM agents. *arXiv preprint arXiv:2504.11703*, 2025. URL <https://arxiv.org/abs/2504.11703>.
- [27] Tobin South, Samuele Marro, Thomas Hardjono, Robert Mahari, Cedric Deslandes Whitney, Dazza Greenwood, Alan Chan, and Alex Pentland. Authenticated delegation and authorized AI agents. *arXiv preprint arXiv:2501.09674*, 2025. URL <https://arxiv.org/abs/2501.09674>.
- [28] Victor Ojewale, Harini Suresh, and Suresh Venkatasubramanian. Audit trails for accountability in large language models. *arXiv preprint arXiv:2601.20727*, 2026. URL <https://arxiv.org/abs/2601.20727>.